长春理工大学
Changchun University of Science and Technology

面向对象程序设计

8. 运算符重载

2022-03-25

提纲

8.1 引言

- 在类对象上使用运算符(运算符重载)
  - 对于某些操作来说，比函数调用要简洁、明了
  - 是运算符符合所在的上下文环境
- 例
  - <<
    - 流提取符，位左移运算符
  - +
    - 应用与多种类型的数值计算 (integers, floats, etc.)
- 讨论什么时候需要操作符重载

8.2 运算符重载的基础

- 类型
  - 基本类型或者用户自定义类型
  - 为用户自定义类型定义已有的运算符的新语义
    - 不可以创造新的运算符
- 重载运算符
  - 为类创建一个**新函数**
  - 函数名格式是： operator 关键字后接运算符
    - Operator + （重载加法运算符的函数名）

8.2 运算符重载的基础

- 在类对象上使用运算符
  - 该运算符必须重载，也有例外：
    - 赋值运算符： =
      - 默认：对象的数据成员逐个赋值
    - 取地址运算符： &
      - 返回对象的地址
    - 他们也可以被重载
- 重载提供的简洁的表达
  - object2 = object1.add(object2);
  - object2 = object2 + object1;

8.3 运算符重载的限制

- 不可以改变：
  - 已有的基本数据类型的操作语义
    - 比如，不可以重新定义整数加法的语义
  - 用算优先级 (计算的顺序)
    - 使用括号强制调整计算顺序
  - 结合律 (left-to-right or right-to-left)
  - 操作数的个数
    - & 是单目运算符, 只接受一个操作数
- 不能创造新的运算符
- 运算符必须显式地重载
  - 重载 + ，与 += 的语义无关

8.3 运算符重载的限制

<table>
<tr><th colspan="8">Operators that can be overloaded</th></tr>
<tr><td>+</td><td>-</td><td>*</td><td>/</td><td>%</td><td>^</td><td>&</td><td>|</td></tr>
<tr><td>~</td><td>!</td><td>=</td><td><</td><td>></td><td>+=</td><td>-=</td><td>*=</td></tr>
<tr><td>/=</td><td>%=</td><td>^=</td><td>&=</td><td>|=</td><td><<</td><td>>></td><td>>>=</td></tr>
<tr><td><<=</td><td>==</td><td>!=</td><td><=</td><td>>=</td><td>&&</td><td>||</td><td>++</td></tr>
<tr><td>--</td><td>->*</td><td>,</td><td>-></td><td>[]</td><td>()</td><td>new</td><td>delete</td></tr>
<tr><td>new[]</td><td>delete[]</td><td></td><td></td><td></td><td></td><td></td><td></td></tr>
</table>

<table>
<tr><th colspan="5">Operators that cannot be overloaded</th></tr>
<tr><td>.</td><td>.*</td><td>::</td><td>?:</td><td>sizeof</td></tr>
</table>

8.4 运算符重载的全局函数和成员函数比较

- 运算符函数
  - 成员函数
    - 用 this 隐式获得实参，获得二元运算符的左操作数 (比如 +)
    - 最左边的操作数必须是运算符的一个类对象
  - 非成员函数
    - 需要通过实参提供所有操作数
    - 可以是来自不同类的对象
    - 必须是友元函数才能访问私有和保护性成员
  - 运算符成员函数在以下情况下被自动调用
    - 二元运算符的左操作数的确是该类的对象
    - 一元运算符的单操作数的确是该类的对象

8.4 运算符重载的全局函数和成员函数比较

- 重载 << 运算符
  - 做操作数是 ostream &
    - 例如 cout << classObject 中的cout对象
  - 类似， >> 的左操作数是 istream &
  - 这两个必须是非成员函数

8.4 运算符重载的全局函数和成员函数比较

- 可交换的运算符
  - 想要 + 可交换
    - “a + b” 和 “b + a” 都工作
  - 假设我们有两个类
  - 只有类对象在左操作数位置时，才能以类成员函数的形式进行运算符重载
    - HugeIntClass + Long int
    - 可以是成员函数
  - 否则需要非成员函数的形式实现
    - Long int + HugeIntClass

8.5 重载流插入运算符和流提取运算符

- << 和 >>
  - 对于基本类型来说，已经是重载过了
  - 也可以处理用户自定义类型
- 例
  - PhoneNumber 类
    - 保存一个电话号码
  - 自动以规定形式输出电话号码
    - (123) 456-7890

8.5 重载流插入运算符和流提取运算符

```
// Overloading the stream-insertion and
// stream-extraction operators.
#include <iostream>

using std::cout;
using std::cin;
using std::endl;
using std::ostream;
using std::istream;

#include <iomanip>

using std::setw;

// PhoneNumber class definition
class PhoneNumber {
   friend ostream &operator<<( ostream&, const PhoneNumber & );
   friend istream &operator>>( istream&, PhoneNumber & );

private:
   char areaCode[ 4 ];  // 3-digit area code and null
   char exchange[ 4 ];  // 3-digit exchange and null
   char line[ 5 ];      // 4-digit line and null

}; // end class PhoneNumber
```

8.5 重载流插入运算符和流提取运算符

```
// overloaded stream-insertion operator; cannot be
// a member function if we would like to invoke it with
// cout << somePhoneNumber;
ostream &operator<<( ostream &output, const PhoneNumber &num )
{
   output << "(" << num.areaCode << ") "
          << num.exchange << "-" << num.line;

   return output;     // enables cout << a << b << c;

} // end function operator<<

// overloaded stream-extraction operator; cannot be
// a member function if we would like to invoke it with
// cin >> somePhoneNumber;
istream &operator>>( istream &input, PhoneNumber &num )
{
   input.ignore();                       // skip (
   input >> setw( 4 ) >> num.areaCode; // input area code
   input.ignore( 2 );                    // skip ) and space
   input >> setw( 4 ) >> num.exchange; // input exchange
   input.ignore();                       // skip dash (-)
   input >> setw( 5 ) >> num.line;     // input line

   return input;      // enables cin >> a >> b >> c;
```

8.5 重载流插入运算符和流提取运算符

```
} // end function operator>>

int main()
{
   PhoneNumber phone; // create object phone

   cout << "Enter phone number in the form (123) 456-7890:\n";

   // cin >> phone invokes operator>> by implicitly issuing
   // the non-member function call operator>>( cin, phone )
   cin >> phone;

   cout << "The phone number entered was: " ;

   // cout << phone invokes operator<< by implicitly issuing
   // the non-member function call operator<<( cout, phone )
   cout << phone << endl;

   return 0;

} // end main
```

```
Enter phone number in the form (123) 456-7890:
(800) 555-1212
The phone number entered was: (800) 555-1212
```

8.6 重载一元运算符

- 重载一元运算符
  - 非静态成员函数，无实参
  - 非成员函数，1 个实参
    - 实参必须是类对象或者是类对象的引用
  - 记住：静态成员函数仅仅能访问静态成员

8.6 重载一元运算符

- 后面的一个例子（8.10）
  - 重载运算符“!”检查空字符串
  - 如果是非静态成员函数，不需要实参
    - !s 等同于成员函数调用：s.operator!()

```
class String {
public:
   bool operator!() const;
   ...
};
```

  - 如果是非成员函数，需要一个实参
    - s! 等同于函数调用 operator!(s)

```
class String {
 friend bool operator!( const String & )
   ...
};
```

8.7 重载二元运算符

- 重载二元运算符
  - 非静态成员函数，1 个实参
  - 非成员函数，2 个实参
    - 一个实参必须是类对象或者是类对象引用
- 后面的例子
  - 非静态成员函数，需要 1 个实参

```
class String {
public:
  const String &operator+=( const String & );
  ...
};
```

  - y += z 等同于 y.operator+=(z)

8.7 重载二元运算符

- 后面的例子:
  - 非成员函数，需要 2 个实参
  - 例:

```
class String {
  friend const String &operator+=(
          String &, const String & );
  ...
};
```

  - y += z 等同于函数调用 operator+=(y, z)

8.8 实例：Array 类

- **C++ 中的数组**
  - 没有边界检查（访问越界）
  - 不能进行有意义的比较，比如 ==, <=, >=, !=
  - 不能复制 (数组名是一个指针常量)
  - 不能整体输入输出，一次处理一个元素
- **例:实现一个 Array 类，能够:**
  - 边界检查
  - 赋值
  - 知道自己的大小
  - 整体输入输出 << 和 >>
  - 比较是否相等 == 和 !=

8.8 实例：Array 类

- 拷贝构造函数
  - 当一个对象被拷贝的时候
    - 用一个已有的对象去初始化另外一个对象
      - Array newArray(oldArray);
  - Array类拷贝构造函数的原型
    - Array(const Array &);
    - 参数必须是引用
      - 否则，当调用拷贝构造函数的时候会触发新的拷贝
      - 死循环

8.8 实例：Array 类

```
 // array1.h
 // Array class for storing arrays of integers.
 #ifndef ARRAY1_H
 #define ARRAY1_H

 #include <iostream>

 using std::ostream;
 using std::istream;

class Array {
   friend ostream &operator<<( ostream &, const Array & );
   friend istream &operator>>( istream &, Array & );

public:
   Array( int = 10 );          // default constructor
   Array( const Array & );     // copy constructor
   ~Array();                   // destructor
   int getSize() const;        // return size

   // assignment operator
   const Array &operator=( const Array & );

   // equality operator
   bool operator==( const Array & ) const;
```

8.8 实例：Array 类

```
   // inequality operator; returns opposite of == operator
   bool operator!=( const Array &right ) const
   {
      return ! ( *this == right ); // invokes Array::operator==

   } // end function operator!=

   // subscript operator for non-const objects returns lvalue
   int &operator[]( int );

   // subscript operator for const objects returns rvalue
   const int &operator[]( int ) const;

private:
   int size; // array size
   int *ptr; // pointer to first element of array

}; // end class Array

#endif
```

8.8 实例：Array 类

```
 // array1.cpp
 // Member function definitions for class Array
 #include <iostream>

 using std::cout;
 using std::cin;
 using std::endl;

 #include <iomanip>

using std::setw;

#include <new>          // C++ standard "new" operator

#include <cstdlib>    // exit function prototype

#include "array1.h"   // Array class definition

// default constructor for class Array (default size 10)
Array::Array( int arraySize )
{
   // validate arraySize
   size = ( arraySize > 0 ? arraySize : 10 );

   ptr = new int[ size ]; // create space for array
```

8.8 实例：Array 类

```
    for ( int i = 0; i < size; i++ )
       ptr[ i ] = 0;           // initialize array

} // end Array default constructor

// copy constructor for class Array;
// must receive a reference to prevent infinite recursion
Array::Array( const Array &arrayToCopy )
   : size( arrayToCopy.size )
{
   ptr = new int[ size ]; // create space for array

   for ( int i = 0; i < size; i++ )
      ptr[ i ] = arrayToCopy.ptr[ i ];  // copy into object

} // end Array copy constructor

// destructor for class Array
Array::~Array()
{
   delete [] ptr;  // reclaim array space

} // end destructor
```

8.8 实例：Array 类

```
// return size of array
int Array::getSize() const
{
   return size;

} // end function getSize
// overloaded assignment operator;
// const return avoids: ( a1 = a2 ) = a3
const Array &Array::operator=( const Array &right )
{
   if ( &right != this ) {  // check for self-assignment

      // for arrays of different sizes, deallocate original
      // left-side array, then allocate new left-side array
      if ( size != right.size ) {
         delete [] ptr;          // reclaim space
         size = right.size;      // resize this object
         ptr = new int[ size ]; // create space for array copy

      } // end inner if

      for ( int i = 0; i < size; i++ )
         ptr[ i ] = right.ptr[ i ];  // copy array into object

   } // end outer if
```

8.8 实例：Array 类

```
    return *this;   // enables x = y = z, for example

} // end function operator=

// determine if two arrays are equal and
// return true, otherwise return false
bool Array::operator==( const Array &right ) const
{
    if ( size != right.size )
        return false;    // arrays of different sizes

    for ( int i = 0; i < size; i++ )

        if ( ptr[ i ] != right.ptr[ i ] )
            return false; // arrays are not equal

    return true;          // arrays are equal

} // end function operator==
```

8.8 实例：Array 类

```
  // overloaded subscript operator for non-const Arrays
  // reference return creates an lvalue
int &Array::operator[]( int subscript )
{
   // check for subscript out of range error
   if ( subscript < 0 || subscript >= size ) {
      cout << "\nError: Subscript " << subscript
           << " out of range" << endl;

      exit( 1 );  // terminate program; subscript out of range

   } // end if

   return ptr[ subscript ]; // reference return

} // end function operator[]

```

8.8 实例：Array 类

```
// overloaded subscript operator for const Arrays
// const reference return creates an rvalue
const int &Array::operator[]( int subscript ) const
{
   // check for subscript out of range error
   if ( subscript < 0 || subscript >= size ) {
      cout << "\nError: Subscript " << subscript
           << " out of range" << endl;
      exit( 1 );  // terminate program; subscript out of range
   } // end if

   return ptr[ subscript ]; // const reference return
} // end function operator[]

// overloaded input operator for class Array;
// inputs values for entire array
istream &operator>>( istream &input, Array &a )
{
   for ( int i = 0; i < a.size; i++ )
      input >> a.ptr[ i ];

   return input;   // enables cin >> x >> y;

} // end function
```

8.8 实例：Array 类

```
// overloaded output operator for class Array
ostream &operator<<( ostream &output, const Array &a )
{
   int i;

   // output private ptr-based array
   for ( i = 0; i < a.size; i++ ) {
      output << setw( 12 ) << a.ptr[ i ];

      if ( ( i + 1 ) % 4 == 0 ) // 4 numbers per row of output
         output << endl;

   } // end for

   if ( i % 4 != 0 )  // end last line of output
      output << endl;

   return output;   // enables cout << x << y;

} // end function operator<<
```

8.8 实例：Array 类

```
// Array class test program.
#include <iostream>

using std::cout;
using std::cin;
using std::endl;

#include "array1.h"

int main()
{
   Array integers1( 7 );  // seven-element Array
   Array integers2;       // 10-element Array by default

   // print integers1 size and contents
   cout << "Size of array integers1 is "
        << integers1.getSize()
        << "\nArray after initialization:\n" << integers1;

   // print integers2 size and contents
   cout << "\nSize of array integers2 is "
        << integers2.getSize()
        << "\nArray after initialization:\n" << integers2;
```

8.8 实例：Array 类

```
   // input and print integers1 and integers2
   cout << "\nInput 17 integers:\n";
   cin >> integers1 >> integers2;

   cout << "\nAfter input, the arrays contain:\n"
        << "integers1:\n" << integers1
        << "integers2:\n" << integers2;

   // use overloaded inequality (!=) operator
   cout << "\nEvaluating: integers1 != integers2\n";

   if ( integers1 != integers2 )
      cout << "integers1 and integers2 are not equal\n";

   // create array integers3 using integers1 as an
   // initializer; print size and contents
   Array integers3( integers1 );  // calls copy constructor

   cout << "\nSize of array integers3 is "
        << integers3.getSize()
        << "\nArray after initialization:\n" << integers3;

```

8.8 实例：Array 类

```
   // use overloaded assignment (=) operator
   cout << "\nAssigning integers2 to integers1:\n";
   integers1 = integers2;  // note target is smaller

   cout << "integers1:\n" << integers1
        << "integers2:\n" << integers2;

   // use overloaded equality (==) operator
   cout << "\nEvaluating: integers1 == integers2\n";

   if ( integers1 == integers2 )
      cout << "integers1 and integers2 are equal\n";

   // use overloaded subscript operator to create rvalue
   cout << "\nintegers1[5] is " << integers1[ 5 ];

   // use overloaded subscript operator to create lvalue
   cout << "\n\nAssigning 1000 to integers1[5]\n";
   integers1[ 5 ] = 1000;
   cout << "integers1:\n" << integers1;
   // attempt to use out-of-range subscript
   cout << "\nAttempt to assign 1000 to integers1[15]" << endl;
   integers1[ 15 ] = 1000;  // ERROR: out of range
   return 0;
} // end main
```

8.8 实例：Array 类

```
Size of array integers1 is 7
Array after initialization:
           0           0           0           0
           0           0           0

Size of array integers2 is 10
Array after initialization:
           0           0           0           0
           0           0           0           0
           0           0

Input 17 integers:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17

After input, the arrays contain:
integers1:
           1           2           3           4
           5           6           7
integers2:
           8           9          10          11
          12          13          14          15
```

8.8 实例：Array 类

```
Evaluating: integers1 != integers2
integers1 and integers2 are not equal

Size of array integers3 is 7
Array after initialization:
           1           2           3           4
           5           6           7

Assigning integers2 to integers1:
integers1:
           8           9          10          11
          12          13          14          15
          16          17
integers2:
           8           9          10          11
          12          13          14          15
          16          17

Evaluating: integers1 == integers2
integers1 and integers2 are equal

integers1[5] is 13
```

8.8 实例：Array 类

```
Assigning 1000 to integers1[5]
integers1:
           8           9          10          11
          12        1000          14          15
          16          17

Attempt to assign 1000 to integers1[15]

Error: Subscript 15 out of range
```

8.9 类型转换

- 类型转换
  - 传统上是指，整型到浮点型之类的转换
  - 用户自定义类型也需要转换
- 类型转换运算符 (转换运算符)
  - 实现以下的转换
    - 一个类到另外一个类
    - 一个类到基本类型 (例如：int, char等)
  - 必须是非静态成员函数
    - 不能是友元
  - 不能指定返回类型
    - 返回类型其实就是对象正要转换成的目标类型

8.9 类型转换

- 例
  - 原型
    - A::operator char *() const;
    - 强制转换A类的对象到一个临时的 char* 对象
    - (char *)s 这个强制转换实际上是一个函数调用：

      s.operator char*()
  - 当然也可以是：
    - A::operator int() const;
    - A::operator OtherClass() const;

8.9 类型转换

- 强制类型转换可以避免一些重载的
  - 假设String 可以强制类型转换为： char *
  - cout << s; // s is a String
    - 编译器隐式地将 s 转换为 char *
    - 而不需要重载 << 运算符
  - 编译器只能做一次直接的类型强制转换
    - 不会进行间接的强制类型转换

8.10 实例: String 类

- 构建自己的String类
  - 实现字符串的创建和操作
  - 在标准库中有string类
- 类型转换构造函数
  - 单个实参的构造函数
  - 将别的类型的对象转换成当前的类对象
    - String s1(“hi”);
    - 从一个 char * 的对象创建一个String 的对象
  - 任何一个单参数的构造函数都可以看作是一个类型转换

8.10 实例: String 类

```
 // string1.h
 // String class definition.
 #ifndef STRING1_H
 #define STRING1_H

 #include <iostream>

 using std::ostream;
 using std::istream;

class String {
   friend ostream &operator<<( ostream &, const String & );
   friend istream &operator>>( istream &, String & );

public:
   String( const char * = "" ); // conversion/default ctor
   String( const String & );    // copy constructor
   ~String();                   // destructor

   const String &operator=( const String & ); // assignment
   const String &operator+=( const String & ); // concatenation

   bool operator!() const;                   // is String empty?
   bool operator==( const String & ) const;  // test s1 == s2
   bool operator<( const String & ) const;   // test s1 < s2
```

支持：S+="abc"
为什么？

8.10 实例: String 类

```
    // test s1 != s2
    bool operator!=( const String & right ) const
    {
       return !( *this == right );
    } // end function operator!=

    // test s1 > s2
    bool operator>( const String &right ) const
    {
       return right < *this;
    } // end function operator>

    // test s1 <= s2
    bool operator<=( const String &right ) const
    {
       return !( right < *this );
    } // end function operator <=

    // test s1 >= s2
    bool operator>=( const String &right ) const
    {
       return !( *this < right );

    } // end function operator>=
```

8.10 实例: String 类

```
    char &operator[]( int );                  // subscript operator
    const char &operator[]( int ) const; // subscript operator

    String operator()( int, int );            // return a substring

    int getLength() const;                    // return string length

private:
    int length;   // string length
    char *sPtr;   // pointer to start of string

    void setString( const char * );  // utility function

}; // end class String

#endif
```

可以有任意的操作数

8.10 实例: String 类

```
// string1.cpp
// Member function definitions for class String.
#include <iostream>
using std::cout;
using std::endl;
#include <iomanip>

using std::setw;

#include <new>          // C++ standard "new" operator

#include <cstring>      // strcpy and strcat prototypes
#include <cstdlib>      // exit prototype

#include "string1.h"    // String class definition

// conversion constructor converts char * to String
String::String( const char *s )
   : length( strlen( s ) )
{
   cout << "Conversion constructor: " << s << '\n';
   setString( s );          // call utility function

} // end String conversion constructor
```

8.10 实例: String 类

```
// copy constructor
String::String( const String &copy )
   : length( copy.length )
{
   cout << "Copy constructor: " << copy.sPtr << '\n';
   setString( copy.sPtr ); // call utility function
} // end String copy constructor

// destructor
String::~String()
{
   cout << "Destructor: " << sPtr << '\n';
   delete [] sPtr;            // reclaim string
} // end ~String destructor

// overloaded = operator; avoids self assignment
const String &String::operator=( const String &right )
{
   cout << "operator= called\n";
   if ( &right != this ) {           // avoid self assignment
      delete [] sPtr;                // prevents memory leak
      length = right.length;         // new String length
      setString( right.sPtr );       // call utility function
   }
```

8.10 实例: String 类

```
      else
         cout << "Attempted assignment of a String to itself\n";

      return *this;   // enables cascaded assignments

   } // end function operator=

   // concatenate right operand to this object and
   // store in this object.
   const String &String::operator+=( const String &right )
   {
      size_t newLength = length + right.length;   // new length
      char *tempPtr = new char[ newLength + 1 ];  // create memory

      strcpy( tempPtr, sPtr );                    // copy sPtr
      strcpy( tempPtr + length, right.sPtr );     // copy right.sPtr

      delete [] sPtr;        // reclaim old space
      sPtr = tempPtr;        // assign new array to sPtr
      length = newLength;    // assign new length to length

      return *this;   // enables cascaded calls

   } // end function operator+=
```

8.10 实例: String 类

```
   // is this String empty?
   bool String::operator!() const
   {
      return length == 0;

   } // end function operator!

   // is this String equal to right String?
   bool String::operator==( const String &right ) const
   {
      return strcmp( sPtr, right.sPtr ) == 0;

   } // end function operator==

   // is this String less than right String?
   bool String::operator<( const String &right ) const
   {
      return strcmp( sPtr, right.sPtr ) < 0;

  } // end function operator<

```

8.10 实例: String 类

```
// return reference to character in String as lvalue
char &String::operator[]( int subscript )
{
   // test for subscript out of range
   if ( subscript < 0 || subscript >= length ) {
      cout << "Error: Subscript " << subscript
           << " out of range" << endl;
      exit( 1 );  // terminate program
   }
   return sPtr[ subscript ];  // creates lvalue

} // end function operator[]

// return reference to character in String as rvalue
const char &String::operator[]( int subscript ) const
{
   // test for subscript out of range
   if ( subscript < 0 || subscript >= length ) {
      cout << "Error: Subscript " << subscript
           << " out of range" << endl;
      exit( 1 );  // terminate program
   }
   return sPtr[ subscript ];  // creates rvalue

} // end function operator[]
```

8.10 实例: String 类

```
// return a substring beginning at index and
// of length subLength
String String::operator()( int index, int subLength )
{
   // if index is out of range or substring length < 0,
   // return an empty String object
   if ( index < 0 || index >= length || subLength < 0 )
      return "";  // converted to a String object automatically

   // determine length of substring
   int len;

   if ( ( subLength == 0 ) || ( index + subLength > length ) )
      len = length - index;
   else
      len = subLength;

   // allocate temporary array for substring and
   // terminating null character
   char *tempPtr = new char[ len + 1 ];

   // copy substring into char array and terminate string
   strncpy( tempPtr, &sPtr[ index ], len );
   tempPtr[ len ] = '\0';
```

8.10 实例: String 类

```
   // create temporary String object containing the substring
   String tempString( tempPtr );
   delete [] tempPtr;  // delete temporary array

   return tempString;  // return copy of the temporary String

} // end function operator()

// return string length
int String::getLength() const
{
   return length;

} // end function getLenth

// utility function called by constructors and operator=
void String::setString( const char *string2 )
{
   sPtr = new char[ length + 1 ]; // allocate memory
   strcpy( sPtr, string2 );       // copy literal to object

} // end function setString
```

8.10 实例: String 类

```
// overloaded output operator
ostream &operator<<( ostream &output, const String &s )
{
   output << s.sPtr;

   return output;   // enables cascading

} // end function operator<<

// overloaded input operator
istream &operator>>( istream &input, String &s )
{
   char temp[ 100 ];   // buffer to store input

   input >> setw( 100 ) >> temp;
   s = temp;          // use String class assignment operator

   return input;    // enables cascading

} // end function operator>>
```

8.10 实例: String 类

```
 // String class test program.
 #include <iostream>

 using std::cout;
 using std::endl;

 #include "string1.h"

int main()
{
   String s1( "happy" );
   String s2( " birthday" );
   String s3;

   // test overloaded equality and relational operators
   cout << "s1 is \"" << s1 << "\"; s2 is \"" << s2
        << "\"; s3 is \"" << s3 << '\"'
        << "\n\nThe results of comparing s2 and s1:"
        << "\ns2 == s1 yields "
        << ( s2 == s1 ? "true" : "false" )
        << "\ns2 != s1 yields "
        << ( s2 != s1 ? "true" : "false" )
        << "\ns2 >  s1 yields "
        << ( s2 > s1 ? "true" : "false" )
```

8.10 实例: String 类

```
           << "\ns2 <  s1 yields "
           << ( s2 < s1 ? "true" : "false" )
           << "\ns2 >= s1 yields "
           << ( s2 >= s1 ? "true" : "false" )
           << "\ns2 <= s1 yields "
           << ( s2 <= s1 ? "true" : "false" );

      // test overloaded String empty (!) operator
      cout << "\n\nTesting !s3:\n";

      if ( !s3 ) {
         cout << "s3 is empty; assigning s1 to s3;\n";
         s3 = s1;  // test overloaded assignment
         cout << "s3 is \"" << s3 << "\"";
      }

      // test overloaded String concatenation operator
      cout << "\n\ns1 += s2 yields s1 = ";
      s1 += s2;  // test overloaded concatenation
      cout << s1;

      // test conversion constructor
      cout << "\n\ns1 += \" to you\" yields\n";
      s1 += " to you";  // test conversion constructor
      cout << "s1 = " << s1 << "\n\n";
```

8.10 实例: String 类

```

   // test overloaded function call operator () for substring
   cout << "The substring of s1 starting at\n"
        << "location 0 for 14 characters, s1(0, 14), is:\n"
        << s1( 0, 14 ) << "\n\n";

   // test substring "to-end-of-String" option
   cout << "The substring of s1 starting at\n"
        << "location 15, s1(15, 0), is: "
        << s1( 15, 0 ) << "\n\n";  // 0 is "to end of string"

   // test copy constructor
   String *s4Ptr = new String( s1 );
   cout << "\n*s4Ptr = " << *s4Ptr << "\n\n";

   // test assignment (=) operator with self-assignment
   cout << "assigning *s4Ptr to *s4Ptr\n";
   *s4Ptr = *s4Ptr;  // test overloaded assignment
   cout << "*s4Ptr = " << *s4Ptr << '\n';

   // test destructor
   delete s4Ptr;

```

8.10 实例: String 类

```
    // test using subscript operator to create lvalue
    s1[ 0 ] = 'H';
    s1[ 6 ] = 'B';
    cout << "\ns1 after s1[0] = 'H' and s1[6] = 'B' is: "
         << s1 << "\n\n";

    // test subscript out of range
    cout << "Attempt to assign 'd' to s1[30] yields:" << endl;
    s1[ 30 ] = 'd';      // ERROR: subscript out of range

    return 0;

} // end main
```

8.10 实例: String 类

```
Conversion constructor: happy
Conversion constructor:  birthday
Conversion constructor:
s1 is "happy"; s2 is " birthday"; s3 is ""

The results of comparing s2 and s1:
s2 == s1 yields false
s2 != s1 yields true
s2 >  s1 yields false
s2 <  s1 yields true
s2 >= s1 yields false
s2 <= s1 yields true

Testing !s3:
s3 is empty; assigning s1 to s3;
operator= called
s3 is "happy"

s1 += s2 yields s1 = happy birthday

s1 += " to you" yields
Conversion constructor:  to you
Destructor:  to you
s1 = happy birthday to you
```

8.10 实例: String 类

```
Conversion constructor: happy birthday
Copy constructor: happy birthday
Destructor: happy birthday
The substring of s1 starting at
location 0 for 14 characters, s1(0, 14), is:
happy birthday

Destructor: happy birthday
Conversion constructor: to you
Copy constructor: to you
Destructor: to you
The substring of s1 starting at
location 15, s1(15, 0), is: to you

Destructor: to you
Copy constructor: happy birthday to you

*s4Ptr = happy birthday to you

assigning *s4Ptr to *s4Ptr
operator= called
Attempted assignment of a String to itself
*s4Ptr = happy birthday to you
Destructor: happy birthday to you
```

8.10 实例: String 类

```
s1 after s1[0] = 'H' and s1[6] = 'B' is: Happy Birthday to you

Attempt to assign 'd' to s1[30] yields:
Error: Subscript 30 out of range
```

8.11 重载 ++ 和 --

- 自增/自减运算符可以被重载
  - 将Date的一个对象d1的天数自增 1
  - 原型 (成员函数)
    - Date &operator++();
    - ++d1 等同于 d1.operator++()
  - 原型 (非成员函数)
    - Friend Date &operator++(Date &);
    - ++d1 等同于 operator++(d1)

8.11 重载 ++ 和 --

- **前置和后置自增的区别**
  - 为了区分前置和后置，后自增有一个空参数
    - 整数 0
  - 原型 (成员函数)
    - Date operator++(int);
    - d1++ 等同于 d1.operator++(0)
  - 原型 (非成员函数)
    - friend Date operator++(Data &, int);
    - d1++ 等同于 operator++(d1, 0)
  - 这个整型参数没有名字
    - 函数定义中也不出现

8.11 重载 ++ 和 --

- 返回值
  - 前自增
    - 返回引用 (Date &)
    - 左值 (可以被赋值)
  - 后自增
    - 返回值参
    - 返回一个原有对象数值的临时对象
    - 右值 (不可以放在赋值符的左侧)
- 自减函数类似

8.12 实例: Date 类

- 例：Date 类
  - 重载自增运算符
    - 变更日、月和年
  - 重载 += 运算符
  - 测试闰年的功能
  - 判断给定日期是否为当月最后一天的功能

8.12 实例: Date 类

```
 // date1.h
 // Date class definition.
 #ifndef DATE1_H
 #define DATE1_H
 #include <iostream>

 using std::ostream;

 class Date {
    friend ostream &operator<<( ostream &, const Date & );

public:
   Date( int m = 1, int d = 1, int y = 1900 ); // constructor
   void setDate( int, int, int ); // set the date

   Date &operator++();              // preincrement operator
   Date operator++( int );          // postincrement operator

   const Date &operator+=( int ); // add days, modify object

   bool leapYear( int ) const;    // is this a leap year?
   bool endOfMonth( int ) const;  // is this end of month?
```

8.12 实例: Date 类

```
    private:
        int month;
        int day;
        int year;

        static const int days[];       // array of days per month
        void helpIncrement();          // utility function

    }; // end class Date

    #endif
```

8.12 实例: Date 类

```
 // date1.cpp
 // Date class member function definitions.
 #include <iostream>
 #include "date1.h"

 // initialize static member at file scope;
 // one class-wide copy
 const int Date::days[] =
    { 0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };

// Date constructor
Date::Date( int m, int d, int y )
{
   setDate( m, d, y );

} // end Date constructor

// set month, day and year
void Date::setDate( int mm, int dd, int yy )
{
   month = ( mm >= 1 && mm <= 12 ) ? mm : 1;
   year = ( yy >= 1900 && yy <= 2100 ) ? yy : 1900;

```

8.12 实例: Date 类

```
    // test for a leap year
    if ( month == 2 && leapYear( year ) )
       day = ( dd >= 1 && dd <= 29 ) ? dd : 1;
    else
       day = ( dd >= 1 && dd <= days[ month ] ) ? dd : 1;
 } // end function setDate

 // overloaded preincrement operator
 Date &Date::operator++()
 {
    helpIncrement();
    return *this;  // reference return to create an lvalue
 } // end function operator++

 // overloaded postincrement operator; note that the dummy
 // integer parameter does not have a parameter name
 Date Date::operator++( int )
 {
    Date temp = *this;  // hold current state of object
    helpIncrement();

    // return unincremented, saved, temporary object
    return temp;   // value return; not a reference return
 } // end function operator++
```

8.12 实例: Date 类

```
   // add specified number of days to date
   const Date &Date::operator+=( int additionalDays )
   {
      for ( int i = 0; i < additionalDays; i++ )
         helpIncrement();

      return *this;    // enables cascading

   } // end function operator+=

   // if the year is a leap year, return true;
   // otherwise, return false
   bool Date::leapYear( int testYear ) const
   {
      if ( testYear % 400 == 0 ||
         ( testYear % 100 != 0 && testYear % 4 == 0 ) )
         return true;   // a leap year
      else
         return false;  // not a leap year

   } // end function leapYear
```

8.12 实例: Date 类

```
// determine whether the day is the last day of the month
bool Date::endOfMonth( int testDay ) const
{
   if ( month == 2 && leapYear( year ) )
      return testDay == 29; // last day of Feb. in leap year
   else
      return testDay == days[ month ];

} // end function endOfMonth

// function to help increment the date
void Date::helpIncrement()
{
   // day is not end of month
   if ( !endOfMonth( day ) )
      ++day;

   else

      // day is end of month and month < 12
      if ( month < 12 ) {
         ++month;
         day = 1;
      }
```

8.12 实例: Date 类

```
        // last day of year
        else {
            ++year;
            month = 1;
            day = 1;
        }

} // end function helpIncrement

// overloaded output operator
ostream &operator<<( ostream &output, const Date &d )
{
    static char *monthName[ 13 ] = { "", "January",
        "February", "March", "April", "May", "June",
        "July", "August", "September", "October",
        "November", "December" };

    output << monthName[ d.month ] << ' '
           << d.day << ", " << d.year;

    return output;    // enables cascading

} // end function operator<<
```

8.12 实例: Date 类

```
 // Date class test program.
 #include <iostream>

 using std::cout;
 using std::endl;

 #include "date1.h"  // Date class definition

int main()
{
   Date d1;  // defaults to January 1, 1900
   Date d2( 12, 27, 1992 );
   Date d3( 0, 99, 8045 );  // invalid date

   cout << "d1 is " << d1 << "\nd2 is " << d2
        << "\nd3 is " << d3;

   cout << "\n\nd2 += 7 is " << ( d2 += 7 );

   d3.setDate( 2, 28, 1992 );
   cout << "\n\n  d3 is " << d3;
   cout << "\n++d3 is " << ++d3;

   Date d4( 7, 13, 2002 );
```

8.12 实例: Date 类

```

      cout << "\n\nTesting the preincrement operator:\n"
           << "  d4 is " << d4 << '\n';
      cout << "++d4 is " << ++d4 << '\n';
      cout << "  d4 is " << d4;

      cout << "\n\nTesting the postincrement operator:\n"
           << "  d4 is " << d4 << '\n';
      cout << "d4++ is " << d4++ << '\n';
      cout << "  d4 is " << d4 << endl;

      return 0;

   } // end main
```

8.12 实例: Date 类

```
d1 is January 1, 1900
d2 is December 27, 1992
d3 is January 1, 1900

d2 += 7 is January 3, 1993

  d3 is February 28, 1992
++d3 is February 29, 1992

Testing the preincrement operator:
  d4 is July 13, 2002
++d4 is July 14, 2002
  d4 is July 14, 2002

Testing the postincrement operator:
  d4 is July 14, 2002
d4++ is July 14, 2002
  d4 is July 15, 2002
```

8.13 标准库中的 string 和 vector 类

- C++中的内置类
  - 每个人都可以使用
  - string
    - 类似于前面的String类
  - vector
    - 可以动态修改尺寸的数组
- 重写 String 和 Array 的例子
  - 采用 string 和 vector

8.13 标准库中的 string 和 vector 类

- string 类
  - 头文件<string>, namespace std
  - 支持初始化 string s1(“hi”);
  - 重载 <<
    - cout << s1
  - 重载了关系运算符
    - == != >= > <= <
  - 重载了赋值运算符 =
  - 重载了连接符 (重载+=)

8.13 标准库中的 string 和 vector 类

- string 类
  - 子串函数 substr
    - s1.substr(0, 14);
      - 从位置 0 开始，返回 14 个字符
    - S1.substr(15)
      - 从位置15开始，往后
  - 重载 []
    - 访问一个元素
    - 没有范围检查 (if subscript invalid)
  - at 函数
    - s1.at(10)
    - 返回下标10处的字符
    - 有边界检车
      - 如果无效会结束程序

8.13 标准库中的 string 和 vector 类

```
// Standard library string class test program.
#include <iostream>

using std::cout;
using std::endl;

#include <string>

using std::string;

int main()
{
   string s1( "happy" );
   string s2( " birthday" );
   string s3;

   // test overloaded equality and relational operators
   cout << "s1 is \"" << s1 << "\"; s2 is \"" << s2
        << "\"; s3 is \"" << s3 << '\"'
        << "\n\nThe results of comparing s2 and s1:"
        << "\ns2 == s1 yields "
        << ( s2 == s1 ? "true" : "false" )
        << "\ns2 != s1 yields "
        << ( s2 != s1 ? "true" : "false" )
```

```
              << "\ns2 >  s1 yields "
              << ( s2 > s1 ? "true" : "false" )
              << "\ns2 <  s1 yields "
              << ( s2 < s1 ? "true" : "false" )
              << "\ns2 >= s1 yields "
              << ( s2 >= s1 ? "true" : "false" )
              << "\ns2 <= s1 yields "
              << ( s2 <= s1 ? "true" : "false" );

   // test string member function empty
   cout << "\n\nTesting s3.empty():\n";

   if ( s3.empty() ) {
      cout << "s3 is empty; assigning s1 to s3;\n";
      s3 = s1;  // assign s1 to s3
      cout << "s3 is \"" << s3 << "\"";
   }

   // test overloaded string concatenation operator
   cout << "\n\ns1 += s2 yields s1 = ";
   s1 += s2;  // test overloaded concatenation
   cout << s1;

```

8.13 标准库中的 string 和 vector 类

```
// test overloaded string concatenation operator
// with C-style string
cout << "\n\ns1 += \" to you\" yields\n";
s1 += " to you";
cout << "s1 = " << s1 << "\n\n";

// test string member function substr
cout << "The substring of s1 starting at location 0 for\n"
     << "14 characters, s1.substr(0, 14), is:\n"
     << s1.substr( 0, 14 ) << "\n\n";

// test substr "to-end-of-string" option
cout << "The substring of s1 starting at\n"
     << "location 15, s1.substr(15), is:\n"
     << s1.substr( 15 ) << '\n';

// test copy constructor
string *s4Ptr = new string( s1 );
cout << "\n*s4Ptr = " << *s4Ptr << "\n\n";

// test assignment (=) operator with self-assignment
cout << "assigning *s4Ptr to *s4Ptr\n";
*s4Ptr = *s4Ptr;
cout << "*s4Ptr = " << *s4Ptr << '\n';
```

8.13 标准库中的 string 和 vector 类

```
    // test destructor
    delete s4Ptr;

    // test using subscript operator to create lvalue
    s1[ 0 ] = 'H';
    s1[ 6 ] = 'B';
    cout << "\ns1 after s1[0] = 'H' and s1[6] = 'B' is: "
         << s1 << "\n\n";

    // test subscript out of range with string member function "at"
    cout << "Attempt to assign 'd' to s1.at( 30 ) yields:" << endl;
    s1.at( 30 ) = 'd';      // ERROR: subscript out of range

    return 0;

} // end main
```

8.13 标准库中的 string 和 vector 类

```
s1 is "happy"; s2 is " birthday"; s3 is ""

The results of comparing s2 and s1:
s2 == s1 yields false
s2 != s1 yields true
s2 >  s1 yields false
s2 <  s1 yields true
s2 >= s1 yields false
s2 <= s1 yields true

Testing s3.empty():
s3 is empty; assigning s1 to s3;
s3 is "happy"

s1 += s2 yields s1 = happy birthday

s1 += " to you" yields
s1 = happy birthday to you

The substring of s1 starting at location 0 for
14 characters, s1.substr(0, 14), is:
happy birthday
```

8.13 标准库中的 string 和 vector 类

```
The substring of s1 starting at
location 15, s1.substr(15), is:
to you

*s4Ptr = happy birthday to you

assigning *s4Ptr to *s4Ptr
*s4Ptr = happy birthday to you

s1 after s1[0] = 'H' and s1[6] = 'B' is: Happy Birthday to you

Attempt to assign 'd' to s1.at( 30 ) yields:

abnormal program termination
```

8.13 标准库中的 string 和 vector 类

- vector 类
  - 头文件 <vector>, namespace std
  - 可以存储任何类型
    - vector< int > myArray(10)
  - 函数 size (myArray.size())
  - 重载 []
    - 获得指定的元素, myArray[3]
  - 重载 !=, ==, and =
    - 不等, 相等, 赋值

8.13 标准库中的 string 和 vector 类

```cpp
// Demonstrating standard library class vector.
#include <iostream>

using std::cout;
using std::cin;
using std::endl;

#include <iomanip>

using std::setw;

#include <vector>

using std::vector;

void outputVector( const vector< int > & );
void inputVector( vector< int > & );

int main()
{
   vector< int > integers1( 7 );    // 7-element vector< int >
   vector< int > integers2( 10 );   // 10-element vector< int >

```

8.13 标准库中的 string 和 vector 类

```
    // print integers1 size and contents
    cout << "Size of vector integers1 is "
         << integers1.size()
         << "\nvector after initialization:\n";
    outputVector( integers1 );

    // print integers2 size and contents
    cout << "\nSize of vector integers2 is "
         << integers2.size()
         << "\nvector after initialization:\n";
    outputVector( integers2 );

    // input and print integers1 and integers2
    cout << "\nInput 17 integers:\n";
    inputVector( integers1 );
    inputVector( integers2 );

    cout << "\nAfter input, the vectors contain:\n"
         << "integers1:\n";
    outputVector( integers1 );
    cout << "integers2:\n";
    outputVector( integers2 );

    // use overloaded inequality (!=) operator
    cout << "\nEvaluating: integers1 != integers2\n";
```

8.13 标准库中的 string 和 vector 类

```
   if ( integers1 != integers2 )
      cout << "integers1 and integers2 are not equal\n";

   // create vector integers3 using integers1 as an
   // initializer; print size and contents
   vector< int > integers3( integers1 );  // copy constructor

   cout << "\nSize of vector integers3 is "
        << integers3.size()
        << "\nvector after initialization:\n";
   outputVector( integers3 );


   // use overloaded assignment (=) operator
   cout << "\nAssigning integers2 to integers1:\n";
   integers1 = integers2;

   cout << "integers1:\n";
   outputVector( integers1 );
   cout << "integers2:\n";
   outputVector( integers1 );

```

8.13 标准库中的 string 和 vector 类

```
    // use overloaded equality (==) operator
    cout << "\nEvaluating: integers1 == integers2\n";

    if ( integers1 == integers2 )
       cout << "integers1 and integers2 are equal\n";

    // use overloaded subscript operator to create rvalue
    cout << "\nintegers1[5] is " << integers1[ 5 ];

    // use overloaded subscript operator to create lvalue
    cout << "\n\nAssigning 1000 to integers1[5]\n";
    integers1[ 5 ] = 1000;
    cout << "integers1:\n";
    outputVector( integers1 );

    // attempt to use out of range subscript
    cout << "\nAttempt to assign 1000 to integers1.at( 15 )"
         << endl;
    integers1.at( 15 ) = 1000;  // ERROR: out of range

    return 0;

} // end main
```

8.13 标准库中的 string 和 vector 类

```
 // output vector contents
 void outputVector( const vector< int > &array )
 {
   for ( int i = 0; i < array.size(); i++ ) {
      cout << setw( 12 ) << array[ i ];

      if ( ( i + 1 ) % 4 == 0 ) // 4 numbers per row of output
         cout << endl;

   } // end for

   if ( i % 4 != 0 )
      cout << endl;

} // end function outputVector

// input vector contents
void inputVector( vector< int > &array )
{
   for ( int i = 0; i < array.size(); i++ )
      cin >> array[ i ];

} // end function inputVector
```

8.13 标准库中的 string 和 vector 类

```
Size of vector integers1 is 7
vector after initialization:
           0           0           0           0
           0           0           0

Size of vector integers2 is 10
vector after initialization:
           0           0           0           0
           0           0           0           0
           0           0

Input 17 integers:
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17

After input, the vectors contain:
integers1:
           1           2           3           4
           5           6           7
integers2:
           8           9          10          11
          12          13          14          15
          16          17

Evaluating: integers1 != integers2
integers1 and integers2 are not equal
```

8.13 标准库中的 string 和 vector 类

```
Size of vector integers3 is 7
vector after initialization:
           1           2           3           4
           5           6           7

Assigning integers2 to integers1:
integers1:
           8           9          10          11
          12          13          14          15
          16          17
integers2:
           8           9          10          11
          12          13          14          15
          16          17

Evaluating: integers1 == integers2
integers1 and integers2 are equal

integers1[5] is 13

Assigning 1000 to integers1[5]
integers1:
           8           9          10          11
          12        1000          14          15
          16          17

Attempt to assign 1000 to integers1.at( 15 )

abnormal program termination
```